

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# Load the dataset
df = pd.read_csv("/kaggle/input/world-marriage-dataset/World Marriage
Dataset.csv")

# Display basic info about the dataset
print("Dataset Overview:")
print(df.info())
print("\nFirst 5 Rows:")
print(df.head())

```

Dataset Overview:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 271604 entries, 0 to 271603

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Sr.No.	271604 non-null	int64
1	Country	271604 non-null	object
2	AgeGroup	271604 non-null	object
3	Sex	271604 non-null	object
4	MaritalStatus	271604 non-null	object
5	DataProcess	271604 non-null	object
6	Data Collection (Start Year)	271604 non-null	int64
7	Data Collection (End Year)	271604 non-null	int64
8	Data Source	271604 non-null	object

dtypes: int64(3), object(6)

memory usage: 18.6+ MB

None

First 5 Rows:

	Sr.No.	Country	AgeGroup	Sex	MaritalStatus	DataProcess	\
0	1	Afghanistan	[15-19]	Man	Divorced	Survey	
1	2	Afghanistan	[20-24]	Man	Divorced	Survey	
2	3	Afghanistan	[25-29]	Man	Divorced	Survey	
3	4	Afghanistan	[30-34]	Man	Divorced	Survey	
4	5	Afghanistan	[35-39]	Man	Divorced	Survey	

	Data Collection (Start Year)	Data Collection (End Year)	\
0	1972	1974	
1	1972	1974	
2	1972	1974	
3	1972	1974	
4	1972	1974	

	Data Source
0	National statistics
1	National statistics
2	National statistics
3	National statistics
4	National statistics

```
# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
Missing Values:
Sr.No.          0
Country         0
AgeGroup        0
Sex             0
MaritalStatus   0
DataProcess     0
Data Collection (Start Year)  0
Data Collection (End Year)    0
Data Source     0
dtype: int64
```

```
# Summary statistics
print("\nSummary Statistics:")
print(df.describe())
```

```
Summary Statistics:
      Sr.No.  Data Collection (Start Year)  Data Collection (End
Year)
count  271604.000000                    271604.000000
mean   135802.500000                    1996.961488
std     78405.465594                     14.244590
min      1.000000                    1954.000000
25%     67901.750000                    1986.000000
50%     135802.500000                    2000.000000
```

```

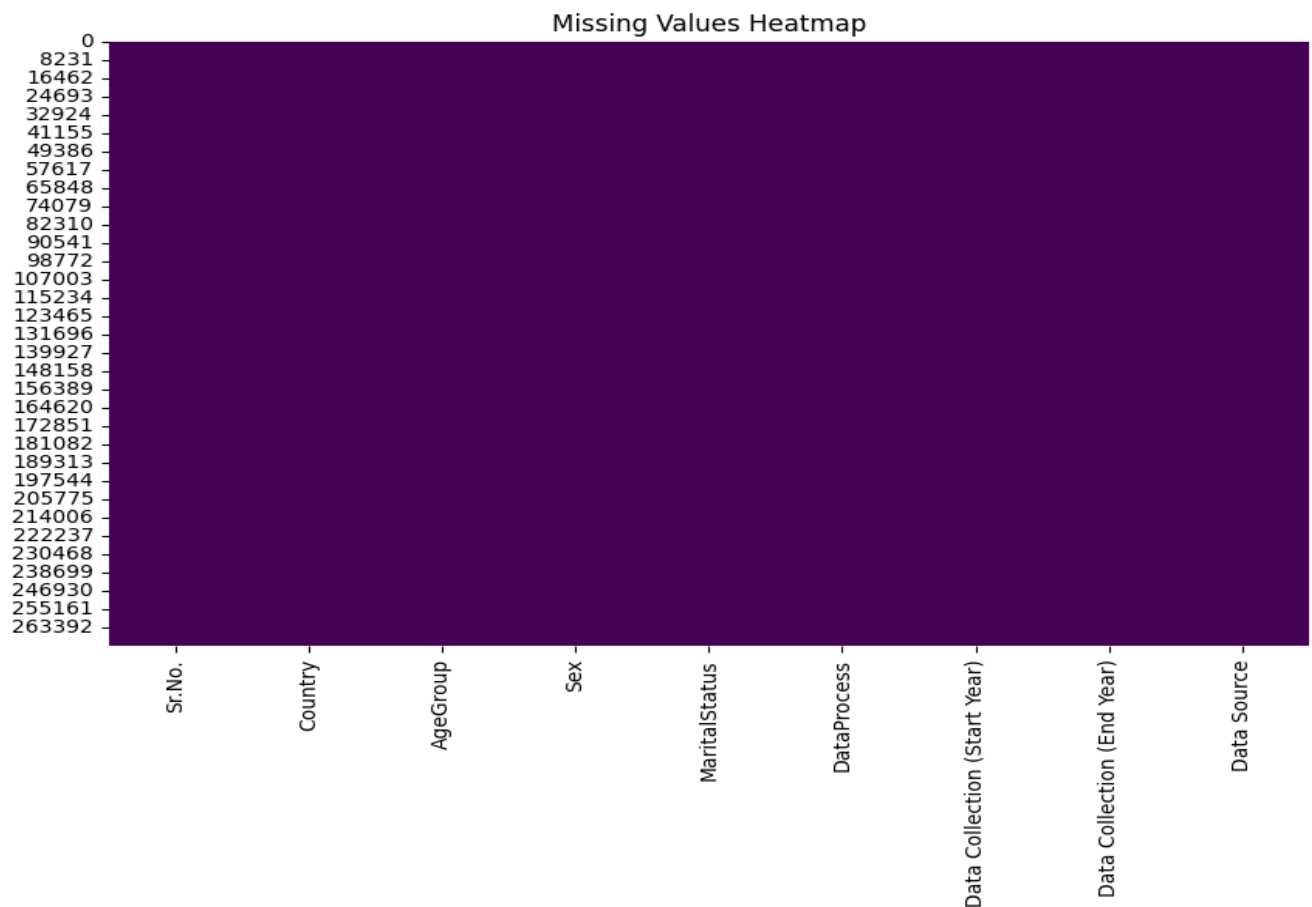
2000.000000
75%      203703.250000      2010.000000
2010.000000
max      271604.000000      2019.000000
2019.000000

```

```

# Visualizing missing values
plt.figure(figsize=(10,6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title("Missing Values Heatmap")
plt.show()

```



```

# Check for duplicate rows
duplicates = df.duplicated().sum()
print(f"\nNumber of duplicate rows: {duplicates}")

```

Number of duplicate rows: 0

```

# Check actual column names
print("\nColumn Names:")
print(df.columns)

```

Column Names:

```
Index(['Sr.No.', 'Country', 'AgeGroup', 'Sex', 'MaritalStatus',  
      'DataProcess',  
      'Data Collection (Start Year)', 'Data Collection (End Year)',  
      'Data Source'],  
      dtype='object')
```

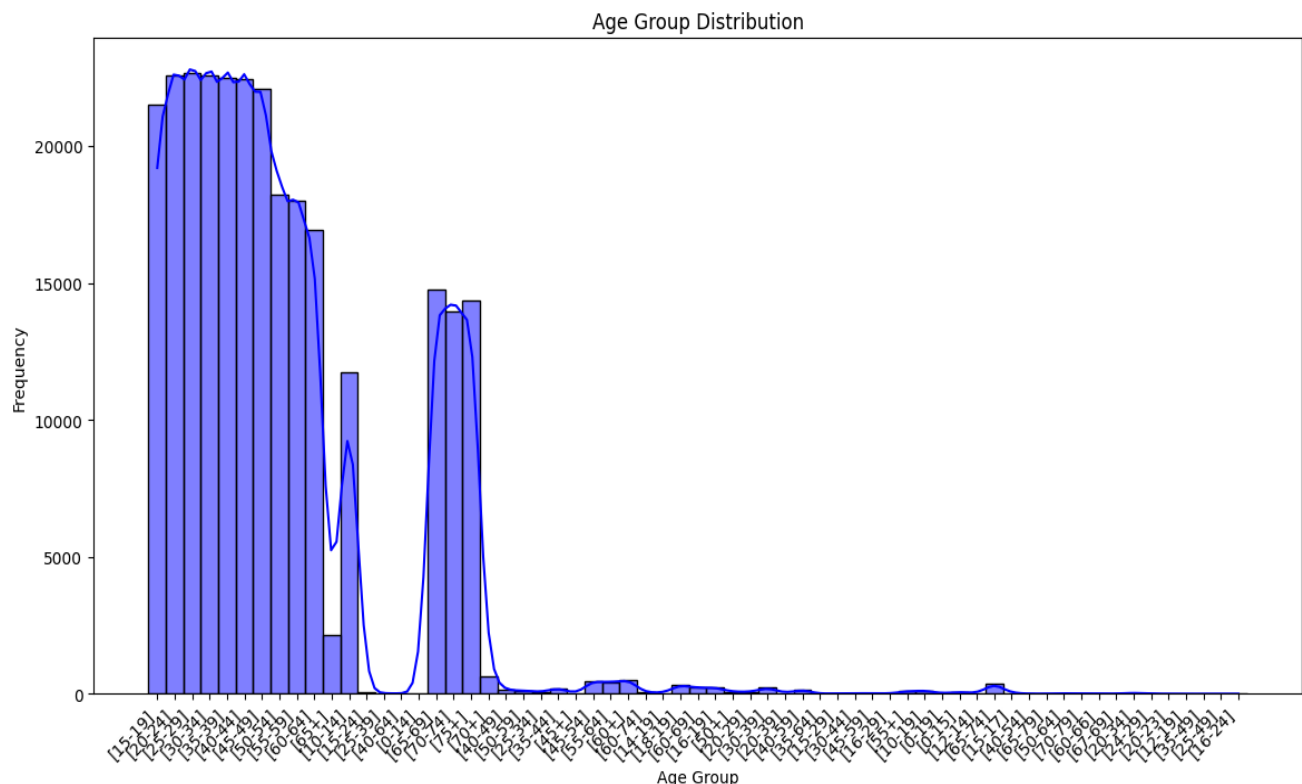
Data distribution plots

```
if 'AgeGroup' in df.columns:  
    plt.figure(figsize=(14,7))  
    sns.histplot(df['AgeGroup'], bins=30, kde=True, color='blue')  
    plt.title("Age Group Distribution")  
    plt.xlabel("Age Group")  
    plt.ylabel("Frequency")  
    plt.xticks(rotation=45, ha='right', fontsize=10)  
    plt.show()  
else:  
    print("\nColumn 'AgeGroup' not found in dataset. Please check the column  
names.")
```

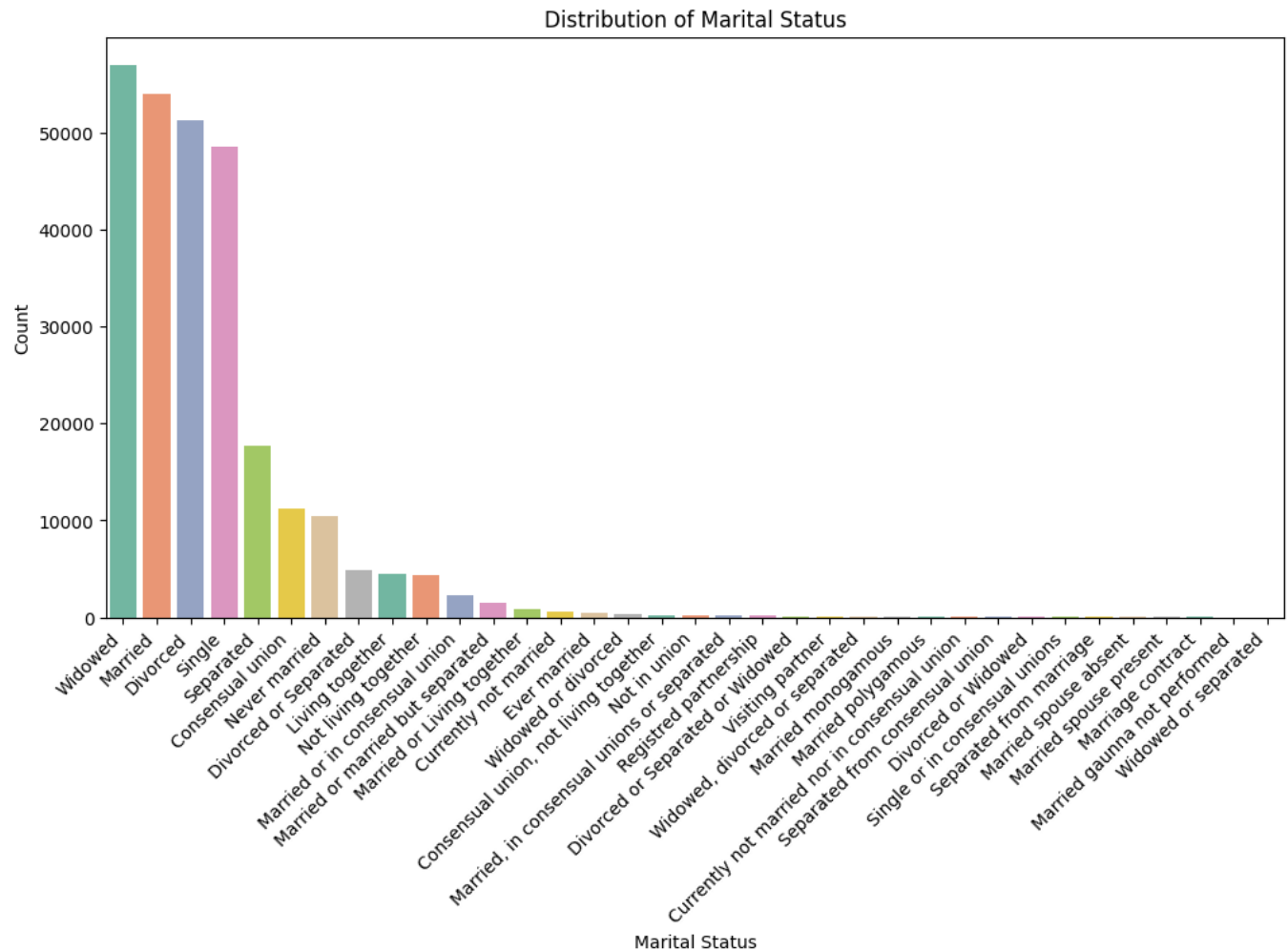
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119:

FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.

with pd.option_context('mode.use_inf_as_na', True):

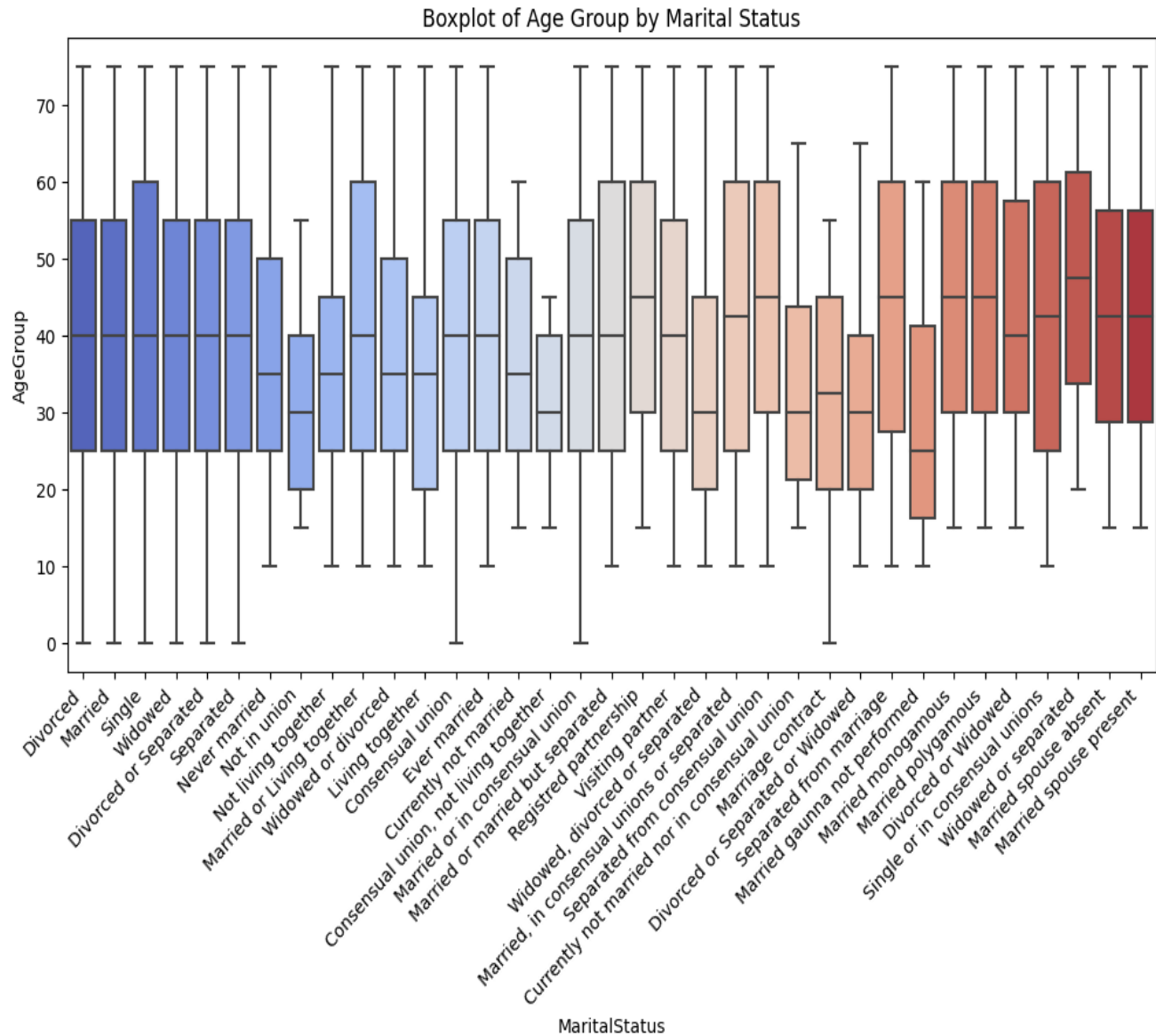


```
# Count of marital status across dataset
plt.figure(figsize=(12,6))
sns.countplot(data=df, x='MaritalStatus',
order=df['MaritalStatus'].value_counts().index, palette='Set2')
plt.title("Distribution of Marital Status")
plt.xlabel("Marital Status")
plt.ylabel("Count")
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.show()
```

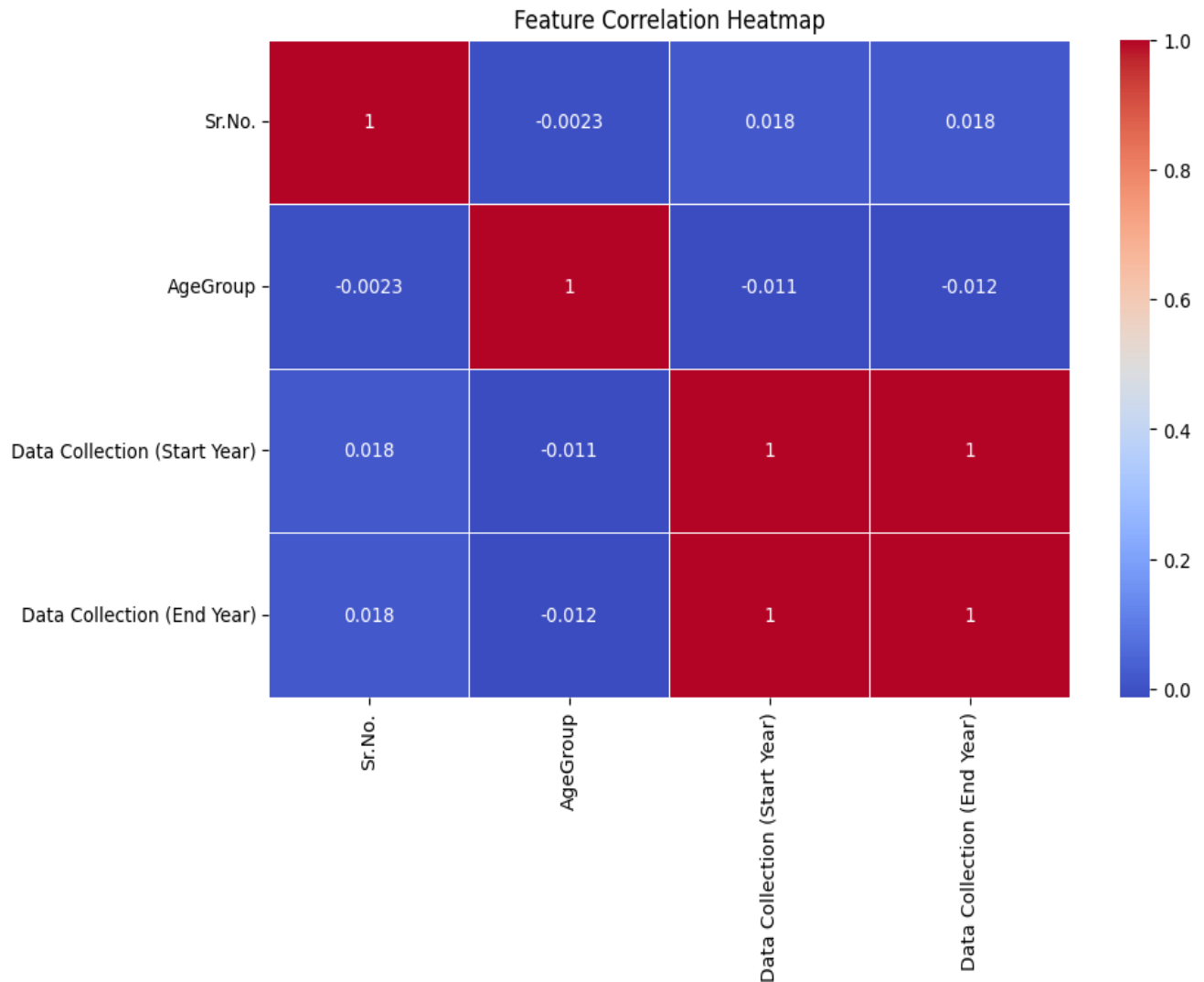


```
# Convert AgeGroup to numerical representation
df['AgeGroup'] =
df['AgeGroup'].astype(str).str.extract('(\d+)').astype(float)

# Boxplots to identify outliers in age and marital status
plt.figure(figsize=(12,6))
sns.boxplot(data=df, x='MaritalStatus', y='AgeGroup', palette='coolwarm')
plt.title("Boxplot of Age Group by Marital Status")
plt.xticks(rotation=45, ha='right', fontsize=10)
plt.show()
```



```
# Heatmap of correlations
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm',
linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```



```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Pairplot
sns.pairplot(df, hue='MaritalStatus')
plt.show()
```

```
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a
future version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass
a length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
  data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
```

FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
```

```
with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
```

```
with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass a length-1 tuple to get_group in a future version of pandas. Pass `(name,)` instead of `name` to silence this warning.
```

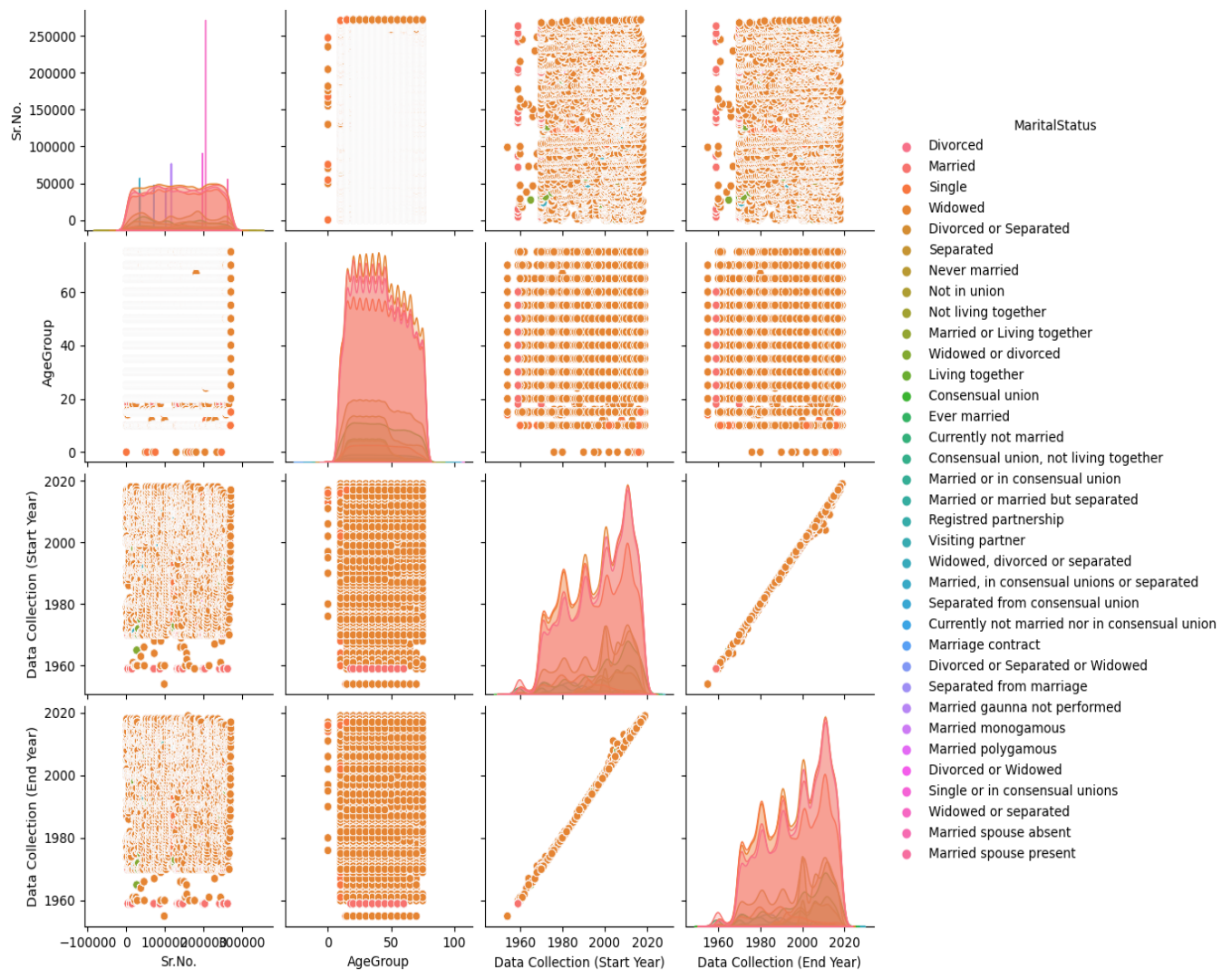
```
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1119:
FutureWarning: use_inf_as_na option is deprecated and will be removed in a future version. Convert inf values to NaN before operating instead.
```



```

with pd.option_context('mode.use_inf_as_na', True):
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass
a length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass
a length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
data_subset = grouped_data.get_group(pd_key)
/usr/local/lib/python3.10/dist-packages/seaborn/_oldcore.py:1075:
FutureWarning: When grouping with a length-1 list-like, you will need to pass
a length-1 tuple to get_group in a future version of pandas. Pass `(name,)`
instead of `name` to silence this warning.
data_subset = grouped_data.get_group(pd_key)

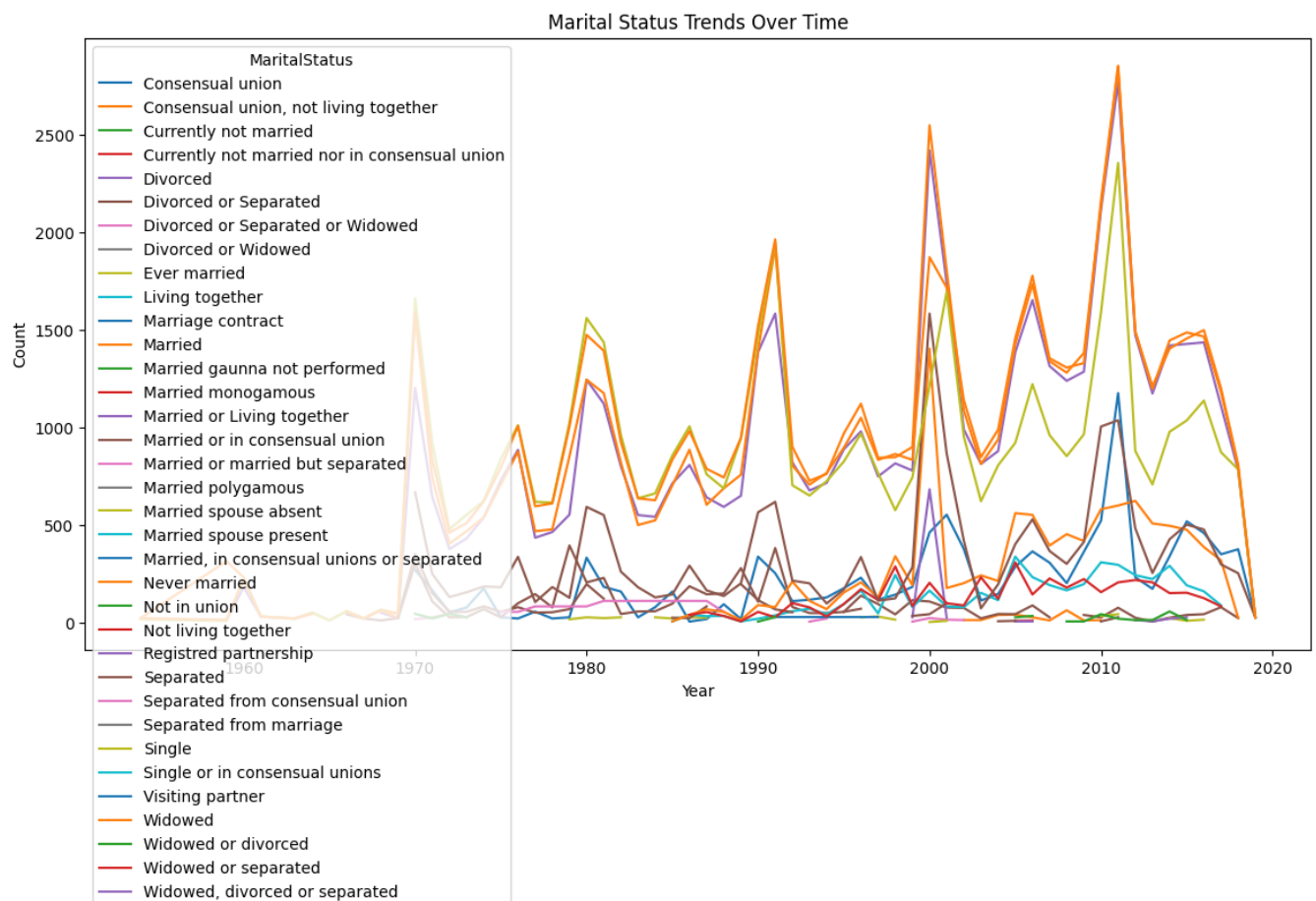
```



```

# Trend Analysis (1970-2019)
df['Data Collection (Start Year)'] = pd.to_numeric(df['Data Collection (Start Year)'], errors='coerce')
df['Data Collection (End Year)'] = pd.to_numeric(df['Data Collection (End Year)'], errors='coerce')
trend_df = df.groupby('Data Collection (Start Year)')['MaritalStatus'].value_counts().unstack()
trend_df.plot(kind='line', figsize=(14,7))
plt.title("Marital Status Trends Over Time")
plt.xlabel("Year")
plt.ylabel("Count")
plt.show()

```



```

print(df.columns)

Index(['Sr.No.', 'Country', 'AgeGroup', 'Sex', 'MaritalStatus',
      'DataProcess',
      'Data Collection (Start Year)', 'Data Collection (End Year)',
      'Data Source'],
      dtype='object')

df.rename(columns={'country': 'Country', 'marital_status': 'MaritalStatus'},
         inplace=True)

```

```

import pandas as pd
df = pd.read_csv("/kaggle/input/world-marriage-dataset/World Marriage
Dataset.csv")
df['Country'] = df['Country'].astype(str)
df['MaritalStatus'] = df['MaritalStatus'].astype(str)

# Compute the number of married and divorced individuals per country
country_stats =
df.groupby('Country')['MaritalStatus'].value_counts().unstack().fillna(0)

# Sort and display the top 10 countries with the highest marriage rates
top_10_married = country_stats.sort_values(by='Married',
ascending=False).head(10)
print("Top 10 Countries with Highest Marriage Rates:")
print(top_10_married[['Married']])

# Sort and display the top 10 countries with the highest divorce rates
top_10_divorced = country_stats.sort_values(by='Divorced',
ascending=False).head(10)
print("\nTop 10 Countries with Highest Divorce Rates:")
print(top_10_divorced[['Divorced']])

# Visualization: Marriage Rates
plt.figure(figsize=(12, 6))
sns.barplot(x=top_10_married.index, y=top_10_married['Married'],
palette='Blues_r')
plt.title("Top 10 Countries with Highest Marriage Rates")
plt.xlabel("Country")
plt.ylabel("Number of Married Individuals")
plt.xticks(rotation=45)
plt.show()

# Visualization: Divorce Rates
plt.figure(figsize=(12, 6))
sns.barplot(x=top_10_divorced.index, y=top_10_divorced['Divorced'],
palette='Reds_r')
plt.title("Top 10 Countries with Highest Divorce Rates")
plt.xlabel("Country")
plt.ylabel("Number of Divorced Individuals")
plt.xticks(rotation=45)
plt.show()

```

Top 10 Countries with Highest Marriage Rates:

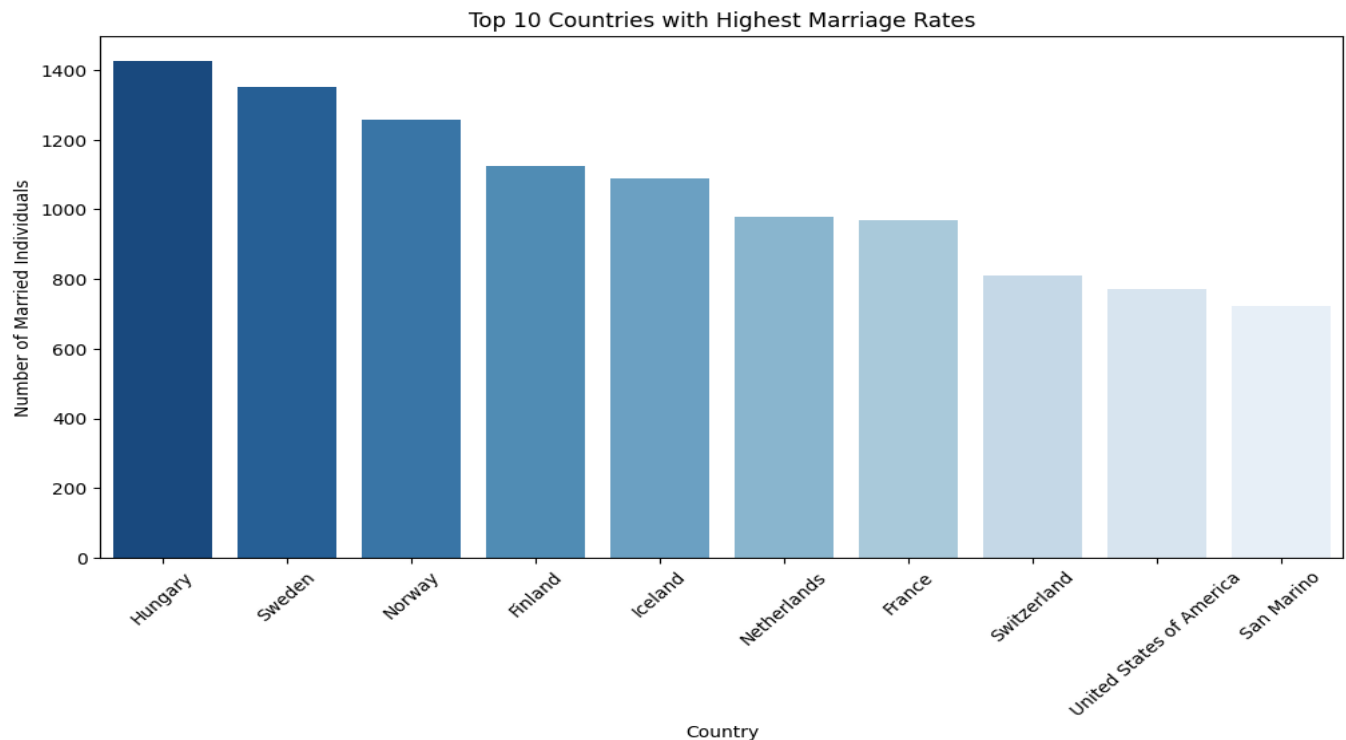
MaritalStatus Married

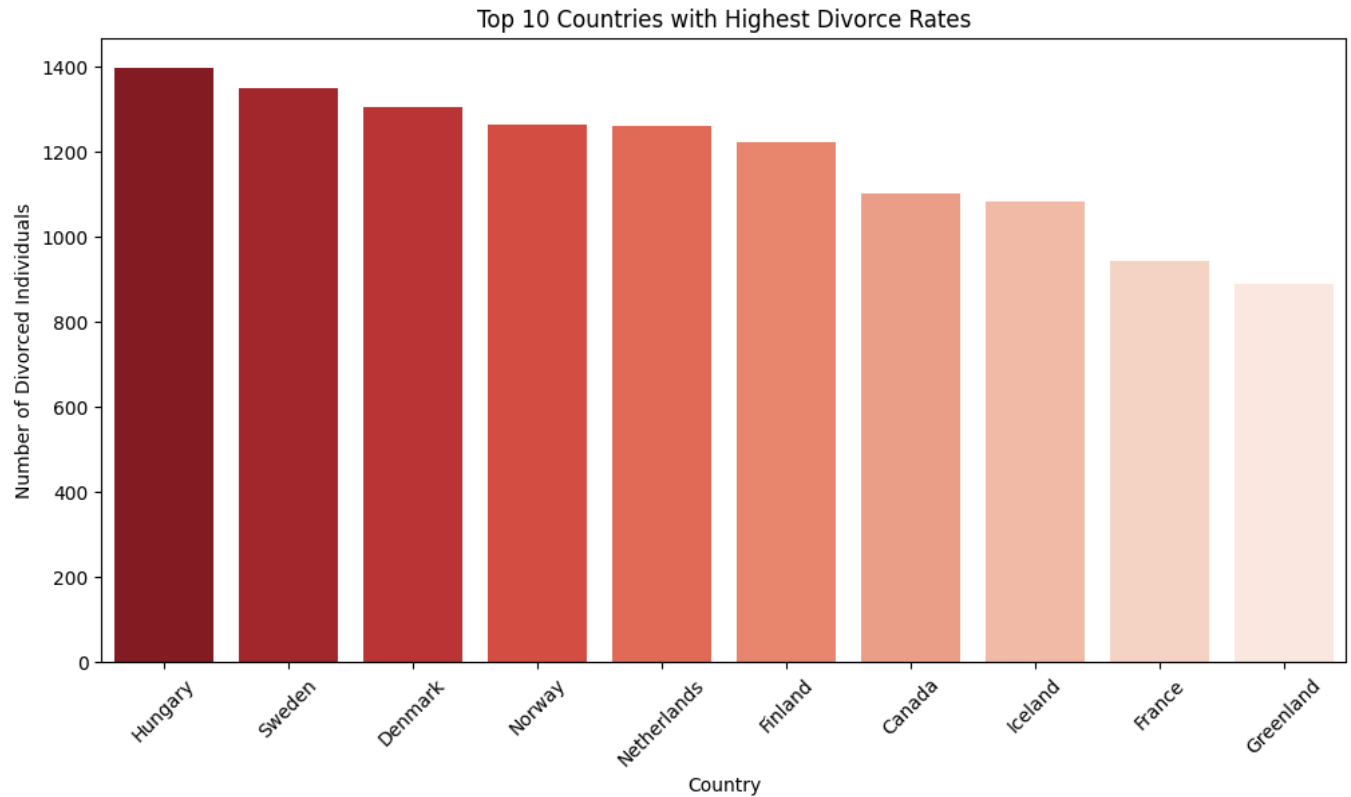
Country	
Hungary	1426.0
Sweden	1351.0
Norway	1258.0
Finland	1126.0
Iceland	1088.0
Netherlands	978.0
France	970.0
Switzerland	809.0
United States of America	770.0
San Marino	723.0

Top 10 Countries with Highest Divorce Rates:

MaritalStatus Divorced

Country	
Hungary	1398.0
Sweden	1351.0
Denmark	1305.0
Norway	1265.0
Netherlands	1260.0
Finland	1224.0
Canada	1104.0
Iceland	1084.0
France	945.0
Greenland	890.0





Top 10 countries with highest and lowest marriage/divorce rates

```
country_stats =
df.groupby('Country')['MaritalStatus'].value_counts().unstack().fillna(0)
print("Top 10 Countries with Highest Marriage Rates:")
print(country_stats.nlargest(10, 'Married'))
print("\nTop 10 Countries with Highest Divorce Rates:")
print(country_stats.nlargest(10, 'Divorced'))
```

Top 10 Countries with Highest Marriage Rates:

MaritalStatus	Consensual union \
Country	
Hungary	344.0
Sweden	264.0
Norway	214.0
Finland	308.0
Iceland	444.0
Netherlands	180.0
France	63.0
Switzerland	194.0
United States of America	110.0
San Marino	28.0

MaritalStatus	Consensual union, not living together \	
Country		
Hungary		0.0
Sweden		0.0
Norway		0.0
Finland		0.0
Iceland		0.0
Netherlands		14.0
France		26.0
Switzerland		0.0
United States of America		0.0
San Marino		0.0

MaritalStatus	Currently not married \	
Country		
Hungary	0.0	
Sweden	0.0	
Norway	0.0	
Finland	0.0	
Iceland	0.0	
Netherlands	0.0	
France	0.0	
Switzerland	0.0	
United States of America	0.0	
San Marino	0.0	

MaritalStatus	Currently not married nor in consensual union \	
Country		
Hungary		0.0
Sweden		0.0
Norway		0.0
Finland		0.0
Iceland		0.0
Netherlands		0.0
France		11.0
Switzerland		0.0
United States of America		0.0
San Marino		0.0

MaritalStatus	Divorced	Divorced or Separated \
Country		
Hungary	1398.0	28.0
Sweden	1351.0	0.0
Norway	1265.0	18.0
Finland	1224.0	0.0
Iceland	1084.0	0.0
Netherlands	1260.0	0.0
France	945.0	0.0
Switzerland	833.0	0.0
United States of America	782.0	0.0

San Marino	772.0	0.0
------------	-------	-----

MaritalStatus	Divorced or Separated or Widowed \
---------------	------------------------------------

Country	
Hungary	0.0
Sweden	0.0
Norway	0.0
Finland	0.0
Iceland	0.0
Netherlands	0.0
France	12.0
Switzerland	0.0
United States of America	0.0
San Marino	0.0

MaritalStatus	Divorced or Widowed	Ever married	Living together
---------------	---------------------	--------------	-----------------

\			
Country			
Hungary	0.0	0.0	0.0
Sweden	0.0	0.0	0.0
Norway	0.0	0.0	0.0
Finland	0.0	0.0	0.0
Iceland	0.0	0.0	0.0
Netherlands	24.0	0.0	0.0
France	0.0	0.0	12.0
Switzerland	0.0	0.0	0.0
United States of America	0.0	0.0	0.0
San Marino	0.0	0.0	0.0

MaritalStatus	... Separated	Separated from consensual union \
---------------	---------------	-----------------------------------

Country	...	
Hungary	54.0	0.0
Sweden	28.0	0.0
Norway	1227.0	0.0
Finland	56.0	0.0
Iceland	1036.0	0.0
Netherlands	0.0	0.0
France	0.0	0.0
Switzerland	112.0	0.0
United States of America	742.0	0.0
San Marino	0.0	0.0

MaritalStatus	Separated from marriage	Single \
---------------	-------------------------	----------

Country		
Hungary	0.0	1424.0
Sweden	0.0	1344.0
Norway	0.0	1292.0
Finland	0.0	1226.0
Iceland	0.0	1090.0

Netherlands	0.0	1284.0
France	0.0	934.0
Switzerland	0.0	838.0
United States of America	0.0	794.0
San Marino	0.0	896.0

MaritalStatus	Single or in consensual unions	Visiting partner \
Country		
Hungary	0.0	0.0
Sweden	0.0	0.0
Norway	0.0	0.0
Finland	0.0	0.0
Iceland	0.0	0.0
Netherlands	0.0	0.0
France	0.0	0.0
Switzerland	0.0	0.0
United States of America	0.0	0.0
San Marino	0.0	0.0

MaritalStatus	Widowed	Widowed or divorced	Widowed or separated
\			
Country			
Hungary	1424.0	0.0	0.0
Sweden	1348.0	0.0	0.0
Norway	1283.0	0.0	0.0
Finland	1223.0	0.0	0.0
Iceland	1081.0	0.0	0.0
Netherlands	1252.0	0.0	0.0
France	945.0	0.0	0.0
Switzerland	828.0	0.0	0.0
United States of America	782.0	12.0	0.0
San Marino	884.0	0.0	0.0

MaritalStatus	Widowed, divorced or separated
Country	
Hungary	0.0
Sweden	0.0
Norway	0.0
Finland	0.0
Iceland	0.0
Netherlands	0.0
France	0.0
Switzerland	0.0
United States of America	0.0
San Marino	0.0

[10 rows x 35 columns]

Top 10 Countries with Highest Divorce Rates:

MaritalStatus	Consensual union	Consensual union, not living together \
Country		
Hungary	344.0	0.0
Sweden	264.0	0.0
Denmark	194.0	0.0
Norway	214.0	0.0
Netherlands	180.0	14.0
Finland	308.0	0.0
Canada	432.0	0.0
Iceland	444.0	0.0
France	63.0	26.0
Greenland	0.0	0.0

MaritalStatus	Currently not married \
Country	
Hungary	0.0
Sweden	0.0
Denmark	0.0
Norway	0.0
Netherlands	0.0
Finland	0.0
Canada	0.0
Iceland	0.0
France	0.0
Greenland	0.0

MaritalStatus	Currently not married nor in consensual union	Divorced \
Country		
Hungary	0.0	1398.0
Sweden	0.0	1351.0
Denmark	0.0	1305.0
Norway	0.0	1265.0
Netherlands	0.0	1260.0
Finland	0.0	1224.0
Canada	0.0	1104.0
Iceland	0.0	1084.0
France	11.0	945.0
Greenland	0.0	890.0

MaritalStatus	Divorced or Separated	Divorced or Separated or Widowed \
Country		
Hungary	28.0	0.0
Sweden	0.0	0.0
Denmark	0.0	0.0
Norway	18.0	0.0
Netherlands	0.0	0.0
Finland	0.0	0.0
Canada	0.0	0.0
Iceland	0.0	0.0
France	0.0	12.0

Greenland	28.0	0.0
-----------	------	-----

MaritalStatus	Divorced or Widowed	Ever married	Living together	...	\
Country				...	
Hungary	0.0	0.0	0.0	...	
Sweden	0.0	0.0	0.0	...	
Denmark	0.0	0.0	0.0	...	
Norway	0.0	0.0	0.0	...	
Netherlands	24.0	0.0	0.0	...	
Finland	0.0	0.0	0.0	...	
Canada	0.0	0.0	0.0	...	
Iceland	0.0	0.0	0.0	...	
France	0.0	0.0	12.0	...	
Greenland	0.0	0.0	0.0	...	

MaritalStatus	Separated	Separated from consensual union	\
Country			
Hungary	54.0	0.0	
Sweden	28.0	0.0	
Denmark	140.0	0.0	
Norway	1227.0	0.0	
Netherlands	0.0	0.0	
Finland	56.0	0.0	
Canada	332.0	0.0	
Iceland	1036.0	0.0	
France	0.0	0.0	
Greenland	56.0	0.0	

MaritalStatus	Separated from marriage	Single	\
Country			
Hungary	0.0	1424.0	
Sweden	0.0	1344.0	
Denmark	0.0	1312.0	
Norway	0.0	1292.0	
Netherlands	0.0	1284.0	
Finland	0.0	1226.0	
Canada	0.0	1154.0	
Iceland	0.0	1090.0	
France	0.0	934.0	
Greenland	0.0	922.0	

MaritalStatus	Single or in consensual unions	Visiting partner	Widowed	\
Country				
Hungary	0.0	0.0	1424.0	
Sweden	0.0	0.0	1348.0	
Denmark	0.0	0.0	1299.0	
Norway	0.0	0.0	1283.0	
Netherlands	0.0	0.0	1252.0	
Finland	0.0	0.0	1223.0	

Canada	0.0	0.0	1104.0
Iceland	0.0	0.0	1081.0
France	0.0	0.0	945.0
Greenland	0.0	0.0	916.0

MaritalStatus	Widowed or divorced	Widowed or separated	\
Country			
Hungary	0.0	0.0	
Sweden	0.0	0.0	
Denmark	0.0	0.0	
Norway	0.0	0.0	
Netherlands	0.0	0.0	
Finland	0.0	0.0	
Canada	36.0	0.0	
Iceland	0.0	0.0	
France	0.0	0.0	
Greenland	0.0	0.0	

MaritalStatus	Widowed, divorced or separated
Country	
Hungary	0.0
Sweden	0.0
Denmark	0.0
Norway	0.0
Netherlands	0.0
Finland	0.0
Canada	0.0
Iceland	0.0
France	0.0
Greenland	0.0

[10 rows x 35 columns]

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv("/kaggle/input/world-marriage-dataset/World Marriage
Dataset.csv") # Uncomment and replace with actual file if needed

# Drop unnecessary columns
df = df.drop(columns=['Sr.No.', 'DataProcess', 'Data Collection (Start
Year)', 'Data Collection (End Year)', 'Data Source'])

# Encode categorical variables
le = LabelEncoder()
```

```

df['MaritalStatus'] = le.fit_transform(df['MaritalStatus'])
df['Sex'] = le.fit_transform(df['Sex'])
df['Country'] = le.fit_transform(df['Country'])
df['AgeGroup'] = le.fit_transform(df['AgeGroup'])

# Define features and target variable
X = df.drop(columns=['MaritalStatus']) # Features (independent variables)
y = df['MaritalStatus'] # Target variable

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train a Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
print("Classification Report:\n", classification_report(y_test, y_pred))

```

Model Accuracy: 0.11137497468750575

Classification Report:

	precision	recall	f1-score	support
0	0.04	0.01	0.02	2260
1	0.00	0.00	0.00	60
2	0.08	0.02	0.03	101
3	0.00	0.00	0.00	13
4	0.10	0.10	0.10	10107
5	0.03	0.00	0.01	987
6	0.00	0.00	0.00	26
7	0.00	0.00	0.00	5
8	0.17	0.01	0.02	103
9	0.00	0.00	0.00	926
10	0.00	0.00	0.00	4
11	0.11	0.13	0.12	10909
12	0.00	0.00	0.00	2
13	0.00	0.00	0.00	10
14	0.00	0.00	0.00	174
15	0.09	0.01	0.01	454
16	0.00	0.00	0.00	323
17	0.00	0.00	0.00	10
18	0.00	0.00	0.00	7
19	0.00	0.00	0.00	5

20	0.00	0.00	0.00	40
21	0.02	0.00	0.00	2046
22	0.00	0.00	0.00	45
23	0.00	0.00	0.00	802
24	0.00	0.00	0.00	41
25	0.05	0.02	0.03	3581
26	0.00	0.00	0.00	3
27	0.00	0.00	0.00	14
28	0.14	0.16	0.15	9646
29	0.00	0.00	0.00	7
30	0.00	0.00	0.00	17
31	0.12	0.17	0.14	11516
32	0.00	0.00	0.00	61
33	0.00	0.00	0.00	6
34	0.00	0.00	0.00	10
accuracy			0.11	54321
macro avg	0.03	0.02	0.02	54321
weighted avg	0.10	0.11	0.10	54321

```

/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))

```

Logistic Regression

```

logreg = LogisticRegression()
logreg.fit(X_train, y_train)
y_pred = logreg.predict(X_test)
print("\nLogistic Regression Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

```

Logistic Regression Accuracy: 0.2119990427274903
      precision    recall  f1-score   support

0         0.00      0.00      0.00      2260
1         0.00      0.00      0.00        60

```

2	0.00	0.00	0.00	101
3	0.00	0.00	0.00	13
4	0.00	0.00	0.00	10107
5	0.00	0.00	0.00	987
6	0.00	0.00	0.00	26
7	0.00	0.00	0.00	5
8	0.00	0.00	0.00	103
9	0.00	0.00	0.00	926
10	0.00	0.00	0.00	4
11	0.00	0.00	0.00	10909
12	0.00	0.00	0.00	2
13	0.00	0.00	0.00	10
14	0.00	0.00	0.00	174
15	0.00	0.00	0.00	454
16	0.00	0.00	0.00	323
17	0.00	0.00	0.00	10
18	0.00	0.00	0.00	7
19	0.00	0.00	0.00	5
20	0.00	0.00	0.00	40
21	0.00	0.00	0.00	2046
22	0.00	0.00	0.00	45
23	0.00	0.00	0.00	802
24	0.00	0.00	0.00	41
25	0.00	0.00	0.00	3581
26	0.00	0.00	0.00	3
27	0.00	0.00	0.00	14
28	0.00	0.00	0.00	9646
29	0.00	0.00	0.00	7
30	0.00	0.00	0.00	17
31	0.21	1.00	0.35	11516
32	0.00	0.00	0.00	61
33	0.00	0.00	0.00	6
34	0.00	0.00	0.00	10
accuracy				0.21 54321
macro avg				0.01 0.03 0.01 54321
weighted avg				0.04 0.21 0.07 54321

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
<https://scikit-learn.org/stable/modules/preprocessing.html>
 Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
 n_iter_i = _check_optimize_result(

```

/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, msg_start, len(result))

# Decision Tree
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
print("\nDecision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print(classification_report(y_test, y_pred_dt))

```

```

Decision Tree Accuracy: 0.1076195209955634
      precision    recall  f1-score   support

0         0.03      0.02      0.02      2260
1         0.00      0.00      0.00         60
2         0.15      0.06      0.09        101
3         0.00      0.00      0.00         13
4         0.10      0.14      0.12      10107
5         0.04      0.01      0.01        987
6         0.00      0.00      0.00         26
7         0.00      0.00      0.00          5
8         0.20      0.03      0.05        103
9         0.03      0.00      0.00        926
10        0.00      0.00      0.00          4
11        0.11      0.14      0.12      10909
12        0.00      0.00      0.00          2
13        0.00      0.00      0.00         10
14        0.00      0.00      0.00        174
15        0.06      0.00      0.01        454
16        0.00      0.00      0.00        323
17        0.00      0.00      0.00         10
18        0.00      0.00      0.00          7
19        0.00      0.00      0.00          5
20        0.00      0.00      0.00         40
21        0.02      0.00      0.00       2046
22        0.00      0.00      0.00         45

```

23	0.00	0.00	0.00	802
24	0.00	0.00	0.00	41
25	0.05	0.02	0.03	3581
26	0.00	0.00	0.00	3
27	0.00	0.00	0.00	14
28	0.14	0.14	0.14	9646
29	0.00	0.00	0.00	7
30	0.00	0.00	0.00	17
31	0.11	0.11	0.11	11516
32	0.00	0.00	0.00	61
33	0.00	0.00	0.00	6
34	0.00	0.00	0.00	10
accuracy				0.11 54321
macro avg				0.03 0.02 0.02 54321
weighted avg				0.10 0.11 0.10 54321

```

/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))

```

Random Forest

```

rf = RandomForestClassifier()
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
print("\nRandom Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print(classification_report(y_test, y_pred_rf))

```

```

Random Forest Accuracy: 0.11166952007510908
      precision    recall  f1-score   support

0         0.03         0.01         0.02         2260
1         0.00         0.00         0.00          60
2         0.10         0.02         0.03         101
3         0.00         0.00         0.00          13
4         0.10         0.10         0.10        10107

```


5	0.01	0.00	0.00	987
6	0.00	0.00	0.00	26
7	0.00	0.00	0.00	5
8	0.17	0.01	0.02	103
9	0.00	0.00	0.00	926
10	0.00	0.00	0.00	4
11	0.11	0.13	0.12	10909
12	0.00	0.00	0.00	2
13	0.00	0.00	0.00	10
14	0.00	0.00	0.00	174
15	0.05	0.00	0.01	454
16	0.00	0.00	0.00	323
17	0.00	0.00	0.00	10
18	0.00	0.00	0.00	7
19	0.00	0.00	0.00	5
20	0.00	0.00	0.00	40
21	0.01	0.00	0.00	2046
22	0.00	0.00	0.00	45
23	0.00	0.00	0.00	802
24	0.00	0.00	0.00	41
25	0.05	0.02	0.03	3581
26	0.00	0.00	0.00	3
27	0.00	0.00	0.00	14
28	0.14	0.16	0.15	9646
29	0.00	0.00	0.00	7
30	0.00	0.00	0.00	17
31	0.12	0.17	0.14	11516
32	0.00	0.00	0.00	61
33	0.00	0.00	0.00	6
34	0.00	0.00	0.00	10
accuracy				0.11 54321
macro avg				0.03 0.02 0.02 54321
weighted avg				0.10 0.11 0.10 54321

```

/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no
predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, msg_start, len(result))
/usr/local/lib/python3.10/dist-
packages/sklearn/metrics/_classification.py:1344: UndefinedMetricWarning:
Precision and F-score are ill-defined and being set to 0.0 in labels with no

```

```
predicted samples. Use `zero_division` parameter to control this behavior.  
_warn_prf(average, modifier, msg_start, len(result))
```

```
# Clustering (K-Means)
```

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
kmeans.fit(X)
```

```
df['Cluster'] = kmeans.labels_
```

```
plt.figure(figsize=(12,6))
```

```
sns.scatterplot(data=df, x='AgeGroup', y='Country', hue='Cluster',
```

```
palette='coolwarm')
```

```
plt.title("K-Means Clustering")
```

```
plt.show()
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/cluster/_kmeans.py:870:
```

```
FutureWarning: The default value of `n_init` will change from 10 to 'auto' in  
1.4. Set the value of `n_init` explicitly to suppress the warning
```

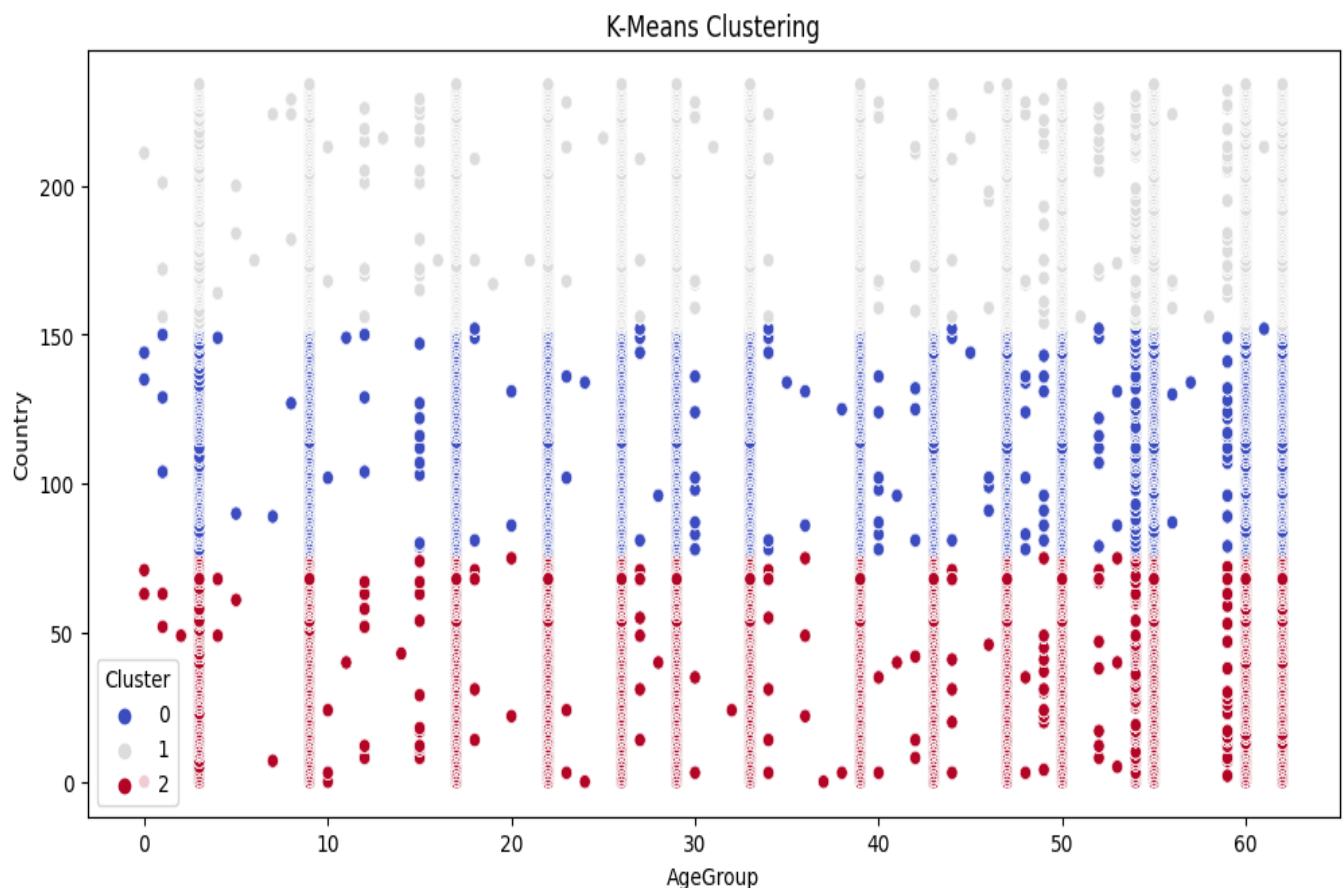
```
warnings.warn(  

```

```
/usr/local/lib/python3.10/dist-packages/IPython/core/pylabtools.py:151:
```

```
UserWarning: Creating legend with loc="best" can be slow with large amounts  
of data.
```

```
fig.canvas.print_figure(bytes_io, **kw)
```



```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import KMeans, DBSCAN

# Load dataset (Ensure the file path is correct)
df = pd.read_csv(r"/kaggle/input/world-marriage-dataset/World Marriage
Dataset.csv")

# Drop unnecessary columns
df = df.drop(columns=['Sr.No.', 'DataProcess', 'Data Collection (Start
Year)', 'Data Collection (End Year)', 'Data Source'])

# Encode categorical variables
le = LabelEncoder()
df['MaritalStatus_encoded'] = le.fit_transform(df['MaritalStatus'])
df['Sex'] = le.fit_transform(df['Sex'])
df['Country'] = le.fit_transform(df['Country'])
df['AgeGroup'] = le.fit_transform(df['AgeGroup'])

# Define features for clustering
X = df[['AgeGroup', 'Sex', 'Country', 'MaritalStatus_encoded']]

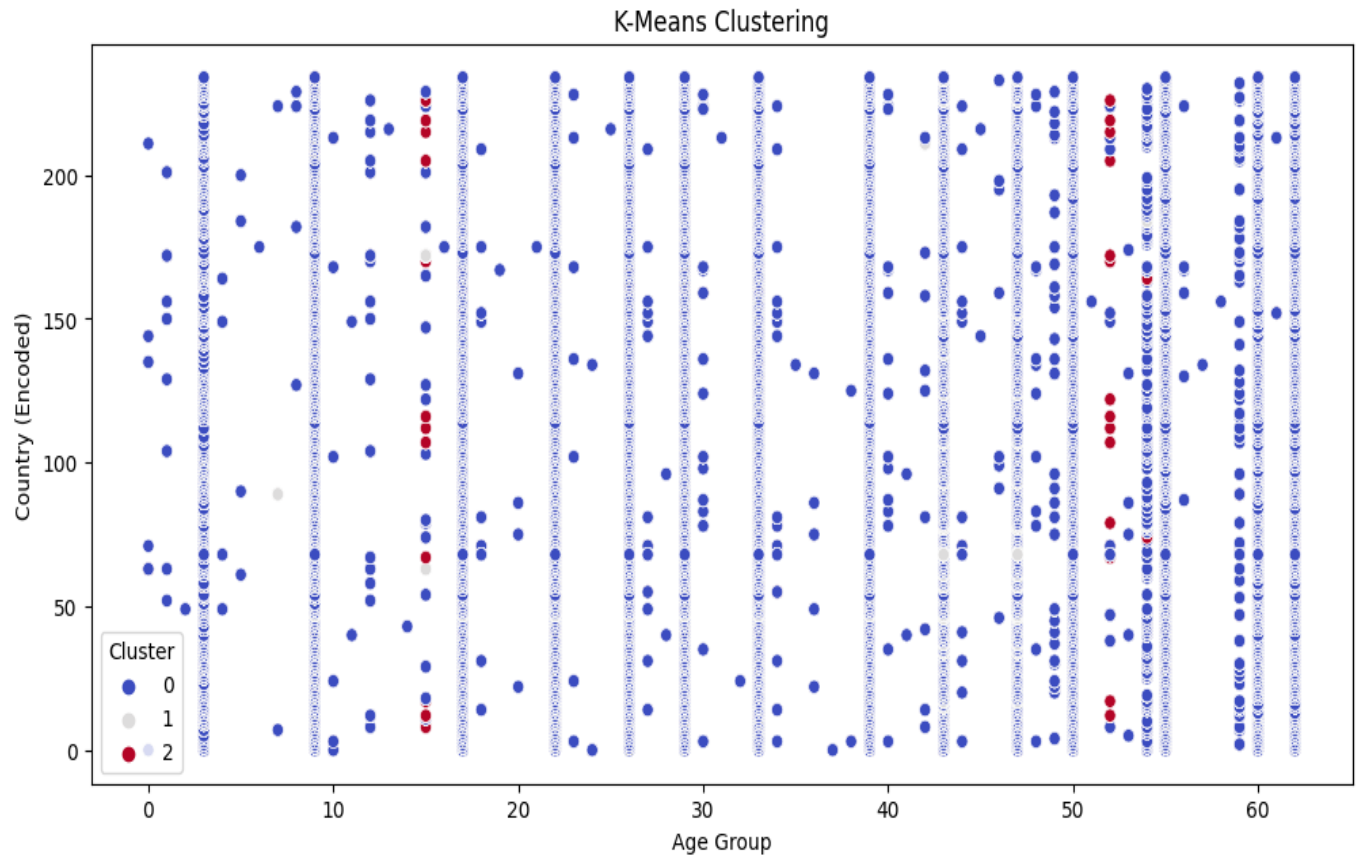
# Scale features (important for clustering)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

### K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)
df['KMeans_Cluster'] = kmeans.labels_

# Visualizing K-Means Clusters
plt.figure(figsize=(12, 6))
sns.scatterplot(data=df, x='AgeGroup', y='Country', hue='KMeans_Cluster',
palette='coolwarm')
plt.title("K-Means Clustering")
plt.xlabel("Age Group")
plt.ylabel("Country (Encoded)")
plt.legend(title="Cluster")
plt.show()

/usr/local/lib/python3.10/dist-packages/IPython/core/pylabtools.py:151:
UserWarning: Creating legend with loc="best" can be slow with large amounts
of data.
  fig.canvas.print_figure(bytes_io, **kw)

```



```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.cluster import DBSCAN

# Load dataset
df = pd.read_csv("/kaggle/input/world-marriage-dataset/World Marriage
Dataset.csv") # Uncomment and replace with actual file if needed

# Drop unnecessary columns
df = df.drop(columns=['Sr.No.', 'DataProcess', 'Data Collection (Start
Year)', 'Data Collection (End Year)', 'Data Source'])

# Encode categorical variables
le = LabelEncoder()
df['MaritalStatus_encoded'] = le.fit_transform(df['MaritalStatus'])
df['Sex'] = le.fit_transform(df['Sex'])
df['Country'] = le.fit_transform(df['Country'])
df['AgeGroup'] = le.fit_transform(df['AgeGroup'])

# Define features for clustering
```

```

X = df[['AgeGroup', 'Sex', 'Country', 'MaritalStatus_encoded']]

# Scale features (important for DBSCAN)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

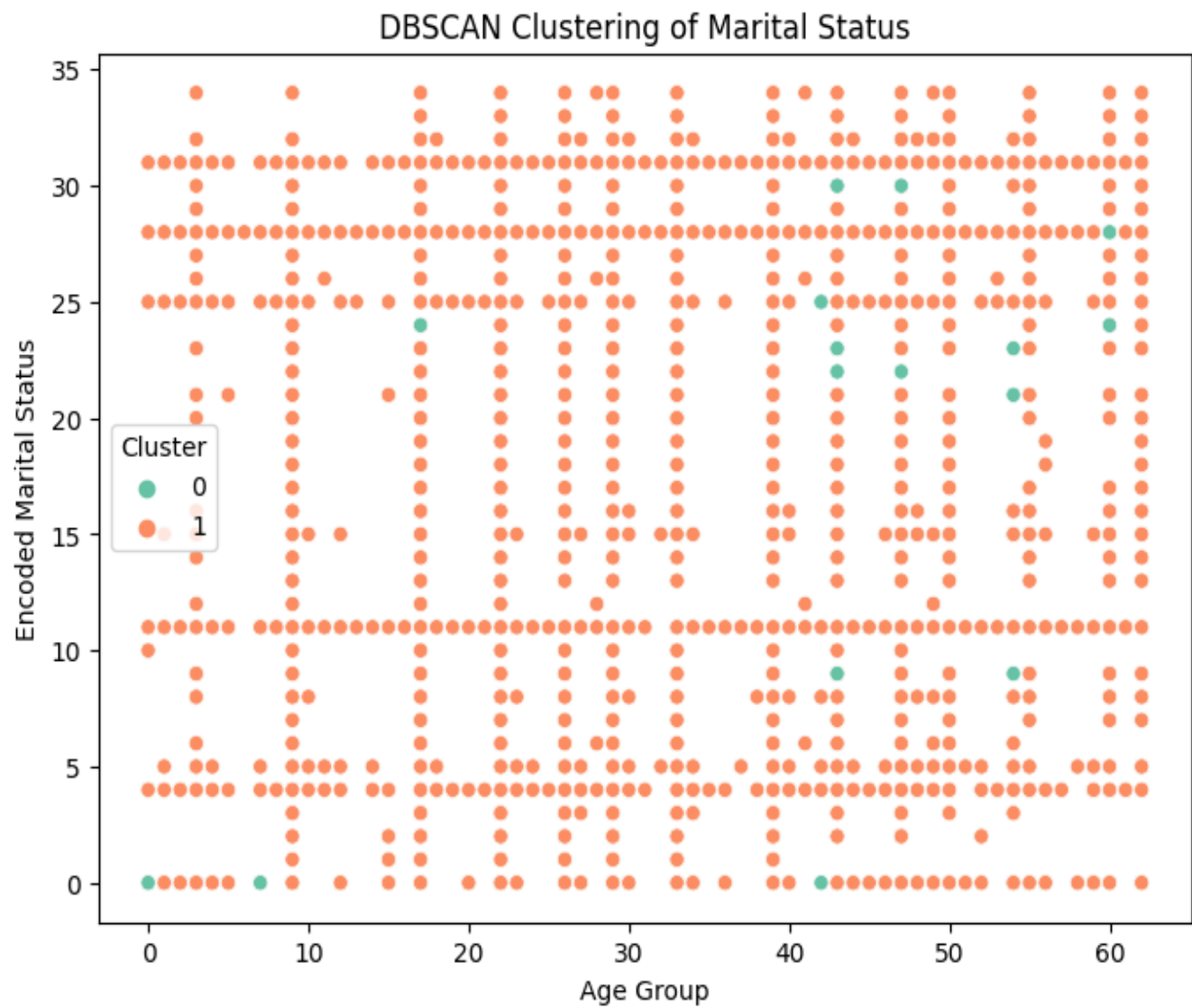
# Apply DBSCAN Clustering
dbscan = DBSCAN(eps=0.5, min_samples=5)
df['DBSCAN_Cluster'] = dbscan.fit_predict(X_scaled)

# Visualizing the clusters
plt.figure(figsize=(8, 6))
sns.scatterplot(x=df['AgeGroup'], y=df['MaritalStatus_encoded'],
hue=df['DBSCAN_Cluster'], palette='Set2')
plt.title("DBSCAN Clustering of Marital Status")
plt.xlabel("Age Group")
plt.ylabel("Encoded Marital Status")
plt.legend(title="Cluster")
plt.show()

print("\n✔Project Analysis Completed - By Nadeem Ahamad")

/usr/local/lib/python3.10/dist-packages/IPython/core/pylabtools.py:151:
UserWarning: Creating legend with loc="best" can be slow with large amounts
of data.
  fig.canvas.print_figure(bytes_io, **kw)

```



✔Project Analysis Completed - By Nadeem Ahamad